

Big Data Programming

Lists, Strings and Nested Lists

Chapter Overview

리스트를 읽고 변경하여 작은 문제를 해결한다

위치와 범위로 값을 선택한다

리스트를 만들고 인덱싱·슬라이싱으로 필요한 값을 읽는다. 그림에서 선택한 위치를 코드와 대응시키며, 원소 하나와 부분 리스트를 구분한다.

변경 전후를 비교한다

추가·삭제·반전·정렬이 같은 목록을 어떻게 바꾸는지 추적한다. 메서드의 반환값과 변경된 원본을 구분하고, 직접 코드를 완성해 결과를 확인한다.

문자열과 2차원 자료로 확장한다

문장을 단어 리스트로 바꾸고, 행과 열을 나누어 읽는다. 오늘의 실습은 앞서 준비한 Python Notebook에서 실행한다.

Lists

여러 값에 하나의 이름을 붙이는 이유

값이 늘어나도 같은 방식으로 처리한다

가격을 price1, price2, price3처럼 각각 저장하면 자료가 늘 때마다 이름과 계산식을 고쳐야 한다. 리스트는 관련 있는 값을 한 묶음으로 관리한다.

리스트의 순서는 위치를 뜻한다

첫 번째 가격과 두 번째 가격을 위치로 구분한다. 순서가 있다는 말은 값이 작은 것부터 자동 정렬된다는 뜻이 아니다.

이름 하나로 전체 자료를 전달한다

prices를 함수의 인수로 넘기거나 반복문에서 차례로 읽을 수 있다. 앞에서 배운 함수와 반복문이 여러 자료를 처리하는 도구로 연결된다.

가격을 리스트에 저장

```
prices = [85000, 140000, 200000]
print(prices)
print(len(prices))
print(sum(prices))
```

실행 결과

```
[85000, 140000, 200000]
3
425000
```

길이와 합계는 다르다

len()은 원소의 개수이고 sum()은 수치 원소의 합계이다.

Lists

그림으로 보기: 여러 이름에서 하나의 목록으로



따로 저장하면 이름이 늘어난다

s0부터 s9까지 각각 이름을 만들면 같은 작업도 여러 번 작성해야 한다. 값의 개수와 코드의 길이가 함께 늘어난다.

리스트에서는 위치를 바꿔 읽는다

scores라는 하나의 이름과 정수 인덱스로 값을 구분한다. 반복문은 위치나 원소를 차례로 처리할 수 있다.

순서와 정렬을 구분한다

그림의 위치는 값이 들어 있는 순서이다. 점수가 작은 순서로 자동 배치된다는 뜻은 아니다.

Lists

대괄호와 쉼표로 리스트 만들기

빈 리스트도 유효한 자료이다

[]는 원소가 하나도 없는 리스트이다. 아직 결과가 없거나, 앞으로 입력받은 값을 모으려는 경우에 사용할 수 있다.

서로 다른 종류의 객체를 담을 수 있다

숫자, 문자열, 다른 리스트를 원소로 넣을 수 있다. 저장이 가능하다는 사실과 모든 원소에 같은 연산이 가능하다는 사실은 구분한다.

원소는 쉼표로 나눈다

[1, "hello", (3 + 2j)]의 길이는 3이다. 복소수 표현식은 계산된 하나의 객체이며, 괄호 안의 기호 수를 원소 수로 세지 않는다.

빈 리스트와 기본 예

```
aa = []
print(len(aa))
aa = [1, 2, 3, 4]
x = [1, "hello", (3 + 2j)]
print(x)
print(len(x))
```

실행 결과

```
0
[1, 'hello', (3+2j)]
3
```

중첩도 하나의 원소이다

[1, [2, 3]]은 바깥 리스트에 원소가 두 개 있다.

Lists

실행 화면: 다양한 값을 같은 구조에 저장하기

1. 리스트 생성과 자료 저장

```
[ ] prices=[85000, 140000, 200000]
    prices
```

```
[85000, 140000, 200000]
```

```
[ ] # 정수형
    integer_list=[1, 2, 3]
    integer_list
```

```
[1, 2, 3]
```

```
[ ] # 실수형
    float_list=[1.0, 2.0, 3.0]
    float_list
```

```
[1.0, 2.0, 3.0]
```

```
[ ] # 문자, 숫자, 실수형 동시 저장
    heterogeneous_list=["string", 0.1, 100, True]
    heterogeneous_list
```

```
['string', 0.1, 100, True]
```

같은 괄호, 다른 원소

가격 목록, 정수 목록, 실수 목록 모두 대괄호와 쉼표로 만든다. 원소의 종류는 실제로 넣은 값에 따라 달라진다.

함께 저장해도 계산은 따로 판단한다

문자열과 숫자를 함께 넣는 것은 가능하다. 그러나 모든 원소를 수치처럼 더하거나 정렬할 수 있다는 뜻은 아니다.

List Indexing

인덱스는 0부터 시작한다

원소의 위치와 순번을 구분한다

letters[0]은 첫 번째 원소 A이다. 원소가 여섯 개이면 양의 인덱스는 0부터 5까지이며, 6은 일곱 번째 위치를 요구한다.

음수 인덱스는 뒤에서 센다

letters[-1]은 마지막 원소 F이다. -2는 E를 가리킨다. 마지막 원소의 위치를 직접 계산하지 않고도 접근할 수 있다.

범위를 벗어난 원소는 읽을 수 없다

letters[6]은 IndexError를 만든다. 값을 찾는 인덱스 연산과 다음에 배울 범위 선택인 슬라이싱은 범위 처리 방식이 다르다.

같은 리스트에서 위치 확인

```
letters = ["A", "B", "C",
           "D", "E", "F"]
print(letters[0])
print(letters[2])
print(letters[-1])
```

실행 결과

```
A
C
F
```

원소	A	B	C	D	E	F
인덱스	0	1	2	3	4	5

List Indexing

Notebook에서 원소와 자료형 확인하기

In [1]:
```
x = [1, "hello", (3 + 2j)]
print(x[2])
print(x[0:2])
```

```
(3+2j)
[1, 'hello']
```

In [2]:
```
letters = ["A", "B", "C", "D", "E", "F"]
print(letters[0], letters[2], letters[-1])
print(letters[0:1], type(letters[0:1]))
```

```
A C F
['A'] <class 'list'>
```

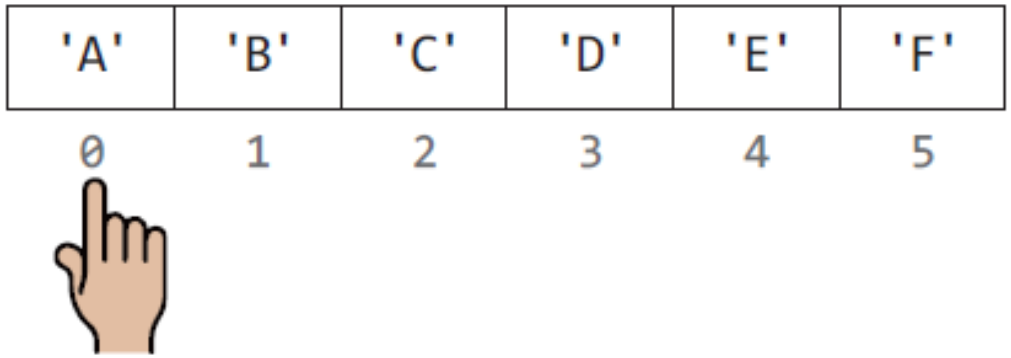
In [3]:
```
letters[6]
```

```
IndexError: list index out of range
```

인덱스 하나로 읽으면 해당 원소가 나온다. 범위를 선택하면 원소가 한 개여도 리스트가 나온다. 마지막 셀의 IndexError는 없는 위치를 읽으려는 의도적인 오류 예이다.

List Indexing

위치 번호와 끝에서 센 번호 대응하기



letters[5]와 letters[-1]

둘 다 마지막 원소 F를 읽는다.
인덱스의 부호가 달라도 같은 위치를 가리킬 수 있다.

같은 원소를 두 가지 위치 번호로 읽는다

원소	A	B	C	D	E	F
앞에서	0	1	2	3	4	5
뒤에서	-6	-5	-4	-3	-2	-1

List Slicing

시작 위치는 포함하고 끝 위치는 제외한다

letters[start:stop]으로 범위를 선택한다

letters[0:3]은 인덱스 0, 1, 2를 고른다. 끝 위치 3은 포함하지 않으므로 결과는 A, B, C가 들어 있는 새 리스트이다.

생략한 경계에는 기본값이 적용된다

[:3]은 처음부터, [3:]은 인덱스 3부터 끝까지 고른다. [:]는 전체 범위를 선택하지만 바깥 리스트는 새로 만들어진다.

원소를 읽는 것과 범위를 읽는 것은 다르다

letters[0]은 문자열 A, letters[0:1]은 ["A"]이다. 슬라이싱의 끝이 길이보다 커도 가능한 범위까지 선택한다.

세 가지 범위 선택

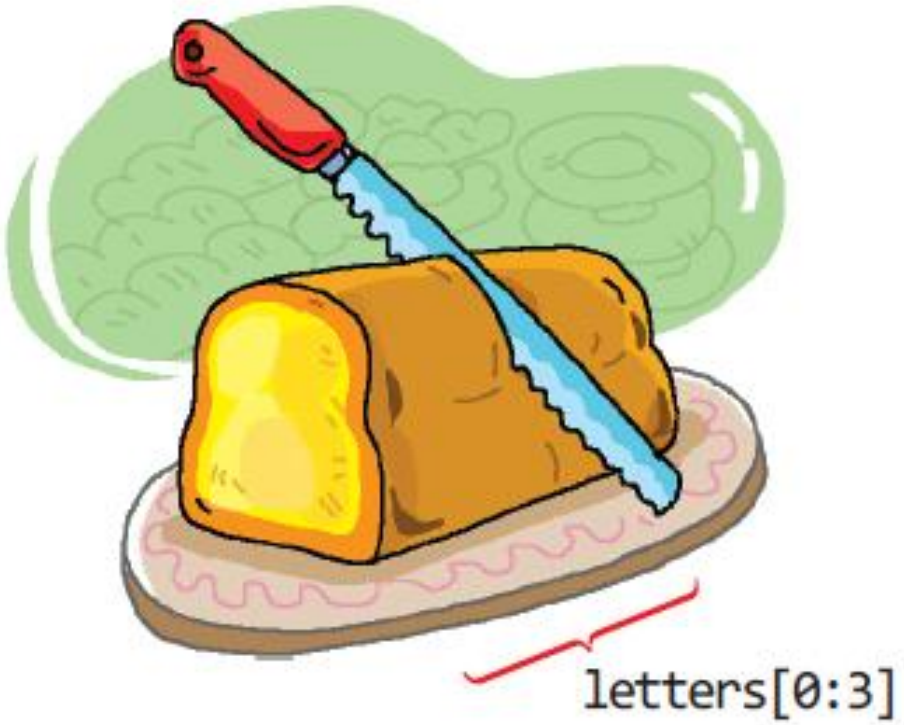
```
letters = ["A", "B", "C",
           "D", "E", "F"]
print(letters[:3])
print(letters[3:])
print(letters[0:1])
print(letters[4:100])
```

실행 결과

```
['A', 'B', 'C']
['D', 'E', 'F']
['A']
['E', 'F']
```

List Slicing

범위를 자를 때 끝 경계는 포함하지 않는다



선택되는 위치를 먼저 적는다

letters[0:3]에서 실제로 선택하는 인덱스는 0, 1, 2이다. 3은 마지막 원소가 아니라 선택을 멈추는 경계이다.

새로운 부분 리스트가 만들어진다

일부를 선택해도 원본 letters에서 A, B, C가 삭제되지는 않는다. 원본을 변경하는 메서드와 구분한다.

원소 하나와 범위를 비교한다

letters[0]은 A라는 문자열이다. letters[0:1]은 A 하나가 들어 있는 리스트이다.

List Slicing

간격과 방향까지 지정하기

세 번째 값은 이동 간격이다

x[start:stop:step]에서 step이 3이면 세 칸씩 이동한다. x[::3]은 인덱스 0, 3, 6, 9를 선택한다.

음수 간격은 뒤에서 앞으로 이동한다

x[10:2:-2]는 유효한 마지막 위치 9부터 두 칸씩 거슬러 올라간다. 끝 경계 2는 제외하므로 선택되는 위치는 9, 7, 5, 3이다.

진행 방향과 끝 경계를 함께 읽는다

음수 step을 쓴다고 자동으로 모든 원소가 나오는 것은 아니다. x[::-1]은 역순 전체를 고르며, step에 0은 사용할 수 없다.

인덱스와 값이 같은 리스트

```
x = [0, 1, 2, 3, 4,
     5, 6, 7, 8, 9]
print(x[::3])
print(x[10:2:-2])
print(x[::-1])
```

실행 결과

```
[0, 3, 6, 9]
[9, 7, 5, 3]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

핵심 확인

슬라이싱은 선택한 원소의 순서로 새 리스트를 만든다.

List Slicing

범위와 이동 간격의 실행 결과 비교

In [1]:
```
x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(x[:3])
print(x[3:])
print(x[::3])
```

```
[0, 1, 2]
[3, 4, 5, 6, 7, 8, 9]
[0, 3, 6, 9]
```

In [2]:
```
print(x[10:2:-2])
print(x[::-1])
print(x)
```

```
[9, 7, 5, 3]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

같은 x에서 시작·끝·간격만 바꾼 결과이다. x[10:2:-2]의 시작 경계 10은 유효한 마지막 위치로 조정된다. x[::-1]을 계산해도 x 자체의 순서는 바뀌지 않는다.

Practice

실습 1. 정류장 목록에서 필요한 구간 선택하기

선택할 위치를 글로 적는다

A부터 F까지 차례로 놓인 정류장 목록이 있다. B에서 시작해 한 정류장씩 건너뛰며 D까지 선택하고, 마지막 두 정류장도 따로 구한다.

결과와 원본을 따로 예상한다

세 번째 결과는 전체 정류장을 역순으로 읽은 목록이다. 슬라이싱 뒤에 stops를 출력하면 어떤 순서인지도 예상한다.

같은 원리를 다른 값에 적용한다

값을 역순으로 읽는 것과 stops 자체를 바꾸는 것은 다르다. 실행 후 각 출력의 자료형과 원본 변화 여부를 설명한다.

완성할 코드

```
stops = ["A", "B", "C",
         "D", "E", "F"]
print(stops[__:__:__])
print(stops[__:])
print(stops[::__])
print(stops)
```

실행 전에 확인할 내용

첫 결과는 B와 D, 두 번째는 E와 F이다. 시작·끝·간격을 각각 어떤 값으로 정해야 하는가?

Practice

해설 1. 선택한 위치와 원본의 순서 확인

In [2]:
```
stops = ["A", "B", "C", "D", "E", "F"]
print(stops[1:5:2])
print(stops[-2:])
print(stops[::-1])
print(stops)
```

```
['B', 'D']
['E', 'F']
['F', 'E', 'D', 'C', 'B', 'A']
['A', 'B', 'C', 'D', 'E', 'F']
```

[1:5:2]는 인덱스 1과 3을 선택한다. [-2:]는 마지막 두 원소를 읽고 [::-1]은 새 역순 리스트를 만든다. stops는 그대로이다.

List Operations

연결과 반복은 원소별 계산이 아니다

리스트의 +는 두 목록을 연결한다

aa + bb는 왼쪽 원소 뒤에 오른쪽 원소를 붙인 새 리스트를 만든다. 같은 위치의 숫자를 각각 더하는 연산이 아니다.

리스트의 *는 원소의 배열을 반복한다

aa * 3은 aa에 있는 원소 순서를 세 번 반복한다. [1, 2, 3]의 각 숫자에 3을 곱하여 [3, 6, 9]를 만드는 연산과 구분한다.

결과를 대입하지 않으면 원본은 유지된다

aa + bb를 계산해도 aa와 bb는 그대로이다. 계산한 새 목록을 다시 사용하려면 다른 이름에 대입한다.

연결·반복 예

```
aa = [1, 2, 3]
bb = [4, 5, 6]
print(aa + bb)
print(aa * 3)
print(aa)
```

실행 결과

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 1, 2, 3, 1, 2, 3]
[1, 2, 3]
```

List Operations

개수, 합계, 포함 여부를 구분한다

len()은 바깥 원소의 개수이다

len([1, 2, 3])은 3이다. 리스트 안에 다른 리스트가 있어도 바깥에서는 그 리스트 하나를 원소 하나로 센다.

in은 같은 원소가 있는지 확인한다

3 in aa는 값 3이 있는지 묻는다. 위치가 3인지를 묻는 aa[3]과 다르다. 3과 문자열 "3"도 서로 다른 값이다.

sum()은 계산 가능한 수치 자료에 쓴다

sum([1, 2, 3])은 6이다. 여러 자료형을 담을 수 있는 리스트라고 해서 문자열과 숫자를 임의로 함께 합산할 수 있는 것은 아니다.

길이·포함·합계 확인

```
aa = [1, 2, 3]
print(len(aa))
print(3 in aa)
print("3" in aa)
print(sum(aa))
print(len([1, [2, 3]]))
```

실행 결과

```
3
True
False
6
2
```

List Methods

append()와 insert()로 원소 추가하기

append()는 맨 끝에 하나를 추가한다

aa.append(1)은 기존 리스트 aa를 변경한다. 점 앞의 이름이 변경 대상이고, 괄호 안의 값이 새로 넣을 원소이다.

insert()는 지정한 위치 앞에 넣는다

letters.insert(1, "A-1")은 인덱스 1 앞에 새 원소를 넣는다. 기존 B 이후 원소들의 위치는 한 칸씩 뒤로 이동한다.

변경한 리스트를 다시 확인한다

두 메서드는 리스트를 직접 바꾸며 반환값은 None이다. aa = aa.append(1)처럼 쓰면 aa에 리스트 대신 None이 대입되므로 주의한다.

맨 끝에 추가

```
aa = []
aa.append(1)
aa.append(2)
aa.append(3)
aa.append("4")
aa.append(5.0)
print(aa)
```

출력: [1, 2, 3, '4', 5.0]

위치를 지정하여 추가

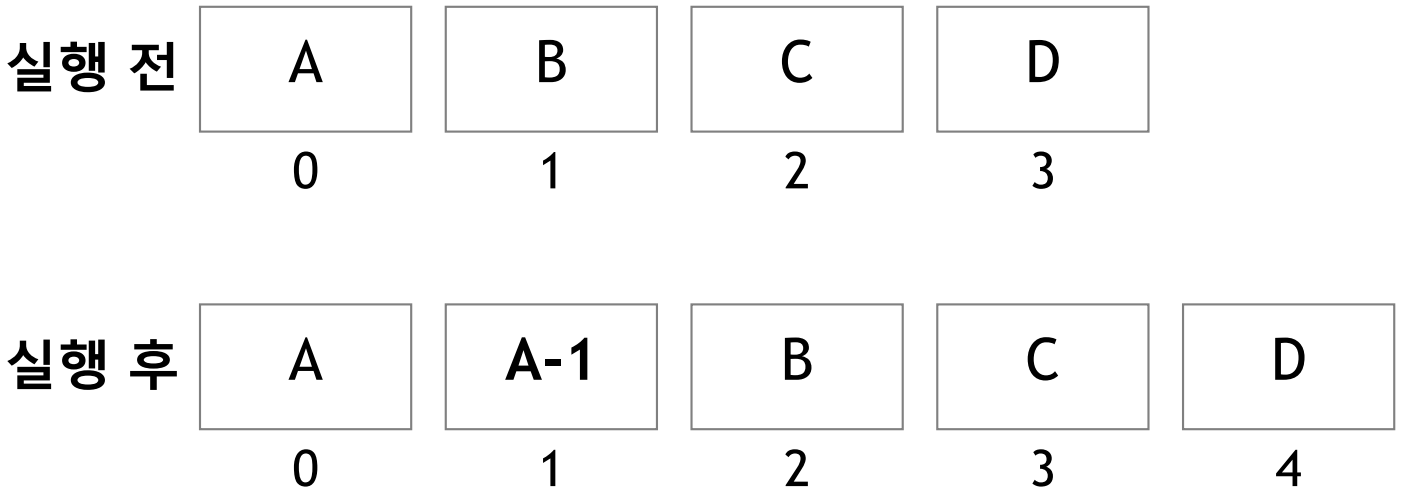
```
letters = ["A", "B", "C"]
letters.insert(1, "A-1")
print(letters)
```

출력: ['A', 'A-1', 'B', 'C']

List Methods

그림으로 보기: 추가하면 뒤 원소의 위치가 이동한다

letters.insert(1, "A-1")



값을 바꾸는 대입과 구분한다

insert는 기존 B를 A-1로 덮어쓰지 않는다. 새 원소가 들어가 길이가 4에서 5로 늘고, B·C·D의 인덱스는 한 칸씩 커진다.

List Methods

extend()와 원소 대입은 다른 작업이다

append()는 목록 전체를 하나로 넣는다

a.append([3, 4])는 리스트 [3, 4] 자체를 원소 하나로 추가한다. 바깥 리스트의 길이는 한 개 늘어난다.

extend()는 전달된 원소들을 차례로 넣는다

b.extend([3, 4])는 3과 4를 각각 추가한다. 합계를 계산하는 함수가 아니라 기존 목록을 늘리는 메서드이다.

인덱스 대입은 기존 위치의 값을 바꾼다

prices[0] = 5000은 첫 번째 원소의 연결을 바꾼다. 새로운 위치를 추가하지 않으므로 리스트의 길이는 그대로이다.

append와 extend 비교

```
a = [1, 2]
b = [1, 2]
a.append([3, 4])
b.extend([3, 4])
print(a)
print(b)
```

출력: [1, 2, [3, 4]]
[1, 2, 3, 4]

가격 수정

```
prices = [85000, 140000, 200000]
prices[0] = 5000
print(prices)
```

출력: [5000, 140000, 200000]

List Methods

추가한 값과 바뀐 길이를 화면에서 확인하기

In [1]:
```
a = [1, 2]
a.append([3, 4])
print(a, len(a))
```

```
[1, 2, [3, 4]] 3
```

In [2]:
```
b = [1, 2]
b.extend([3, 4])
print(b, len(b))
```

```
[1, 2, 3, 4] 4
```

In [3]:
```
prices = [85000, 140000, 200000]
prices[0] = 5000
prices.append(15000)
print(prices, len(prices))
```

```
[5000, 140000, 200000, 15000] 4
```

첫 두 셀은 같은 [1, 2]에서 시작하지만 append와 extend의 결과가 다르다. 마지막 셀은 첫 가격을 수정한 뒤 15000을 추가한 예이다. 대입과 추가를 각각 실행한 시점을 구분한다.

List Methods

값으로 지우기와 위치로 지우기

remove(value)는 일치하는 첫 원소를 지운다

letters.remove("D")는 값 D를 찾는다. 같은 값이 여러 번 있어도 첫 번째만 제거하며, 없는 값을 지우려 하면 ValueError가 발생한다.

del은 지정한 위치를 삭제하는 문장이다

del letters[0]은 첫 번째 원소를 지운다. 삭제 이후에는 뒤에 있던 원소가 앞으로 이동하므로 같은 인덱스가 다른 값을 가리킬 수 있다.

pop()은 제거한 값을 돌려준다

pop()은 기본적으로 마지막 원소를 제거하고 그 값을 반환한다. pop(0)은 첫 원소를 꺼낸다. 빈 리스트에서는 IndexError가 발생한다.

각 예제는 새 리스트에서 시작

```
letters = ["A", "B", "C", "D"]
letters.remove("D")
print(letters)

letters = ["A", "B", "C", "D"]
del letters[0]
print(letters)
last = letters.pop()
print(last, letters)
```

실행 결과

```
['A', 'B', 'C']
['B', 'C', 'D']
D ['B', 'C']
```

List Methods

실행 화면: 같은 값이 두 번 들어 있을 때 삭제

In [4]:
```
data = [10, 20, 20, 30]
data.remove(20)
print(data)
taken = data.pop(1)
print(taken, data)
del data[0]
print(data)
```

```
[10, 20, 30]
20 [10, 30]
[30]
```

remove(20)은 첫 번째 20만 지운다. pop(1)은 그 시점의 1번 위치를 제거해 20을 반환한다. 마지막 del data[0] 뒤에는 [30]만 남는다.

List Methods

reverse()는 현재 순서를 뒤집는다

역순과 내림차순은 같은 뜻이 아니다

reverse()는 이미 들어 있는 순서를 앞뒤로 바꾼다. 크기 비교를 해서 다시 정렬하는 것은 아니므로 숫자가 큰 순서가 된다고 보장할 수 없다.

변경은 같은 리스트에 적용된다

aa.reverse()를 실행하면 aa가 가리키는 리스트의 순서가 바뀐다. aa[::-1]로 새 리스트를 만드는 경우와 원본 변화 여부를 비교한다.

이어서 실행하는 pop()의 결과도 달라진다

[1, 2, 3, 4]를 뒤집으면 마지막 값은 1이다. 이 상태에서 pop()을 호출하면 4가 아니라 1을 꺼낸다.

순서대로 실행

```
aa = [1, 2, 3, 4]
aa.reverse()
print(aa)
print(aa.pop())
print(aa)
```

출력: [4, 3, 2, 1]
1
[4, 3, 2]

정렬되지 않은 자료

```
data = [3, 1, 2]
data.reverse()
print(data)
```

출력: [2, 1, 3]

List Methods

응용 예제: 마지막 방문 기록을 되돌리기

In [6]:
```
history = ["home", "search"]
history.append("detail")
print(history)
last_page = history.pop()
print(last_page)
print(history[-1])
print(history)
```

```
['home', 'search', 'detail']
detail
search
['home', 'search']
```

마지막 기록을 끝에 저장한다

home과 search 다음에 detail 페이지를 방문했다. append로 새 방문을 끝에 기록한다.

pop의 반환값을 사용한다

pop은 detail을 제거하고 돌려준다. 남은 목록의 마지막 원소 search가 돌아갈 위치가 된다.

빈 목록도 생각한다

현재 예제에는 기록이 남아 있다. 빈 리스트에 [-1]이나 pop()을 적용하면 오류가 나므로 실제 사용에서는 길이도 확인한다.

List Sorting

sort()와 sorted()의 차이는 원본과 반환값이다

sort()는 원본 리스트를 정렬한다

letters.sort()는 기존 letters를 오름차순으로 바꾸고 None을 반환한다. letters = letters.sort()로 쓰면 리스트 대신 None에 연결되므로 주의한다.

sorted()는 정렬한 새 리스트를 반환한다

ordered = sorted(letters)는 정렬한 새 리스트를 ordered에 연결한다. 원래 순서와 정렬 결과를 따로 사용할 수 있다.

정렬 기준도 지정할 수 있다

key=abs는 절댓값을 기준으로 비교하고 reverse=True는 내림차순을 요청한다. 서로 비교할 수 없는 자료형을 무조건 섞어 정렬할 수는 없다.

원본을 바꾸는 정렬

```
letters = ["B", "A", "D", "C"]
result = letters.sort()
print(letters)
print(result)
```

실행 결과

```
['A', 'B', 'C', 'D']
None
```

새 리스트를 얻는 정렬

```
x = sorted([-4, 1, -2, 3],
           key=abs, reverse=True)
print(x)
```

실행 결과

```
[-4, 3, -2, 1]
```

List Sorting

반환값과 원본의 순서를 함께 확인하기

In [1]:
```
data = [4, 1, 2, 3]
ordered = sorted(data)
print("data:", data)
print("ordered:", ordered)
```

```
data: [4, 1, 2, 3]
ordered: [1, 2, 3, 4]
```

In [2]:
```
result = data.sort()
print("data:", data)
print("result:", result)
```

```
data: [1, 2, 3, 4]
result: None
```

In [3]:
```
print(sorted([-4, 1, -2, 3], key=abs, reverse=True))
```

```
[-4, 3, -2, 1]
```

첫 셀에서 sorted(data)는 정렬된 새 리스트를 만들고 data는 그대로 남는다. 두 번째 셀에서 data.sort()는 data를 바꾸지만 반환값은 None이다. 출력된 목록과 반환값을 같은 것으로 읽지 않는다.

List Sorting

정렬 예제의 각 셀에서 무엇을 표시하는가

원소 정렬하기: sort 함수 / sorted 함수

```
[ ] x=[4, 1, 2, 3]
    x.sort( )
    x
```

```
[1, 2, 3, 4]
```

```
[ ] y=[4, 1, 2, 3]
    sorted(y)
    y
```

```
[4, 1, 2, 3]
```

```
[ ] x=sorted([-4, 1, -2, 3], key=abs, reverse=True)
    x
```

```
[-4, 3, -2, 1]
```

sort와 sorted의 결과를 나누어 본다

x.sort() 뒤에는 x 자체가 정렬된다. sorted(y)는 새 결과를 만들지만 다시 y를 표시하면 이전 순서가 보인다.

Notebook의 마지막 표현식

두 번째 셀에는 sorted(y)와 y가 연달아 있다. 마지막 표현식 y의 값만 자동 표시되므로 정렬 결과가 없다고 판단하면 안 된다.

절댓값 기준의 내림차순

마지막 예는 절댓값 4, 3, 2, 1의 순서로 정렬한다. 실제 원소의 부호는 그대로 남는다.

Practice

실습: 리스트의 상태를 한 줄씩 추적하기

실행하기 전에 결과를 예상한다

코드를 위에서 아래로 읽고 각 문장 뒤의 리스트를 적는다. append는 끝에 추가하고 insert는 지정 위치 앞에 넣으며 remove는 값을 찾아 삭제한다.

원소의 위치가 달라지는 시점을 찾는다

1번 위치에 A-1을 넣은 뒤 기존 원소의 인덱스가 어떻게 이동하는지 먼저 확인한다. pop의 반환값과 남아 있는 리스트도 따로 적는다.

예상과 실제 결과를 비교한다

마지막 두 출력과 길이를 예상한 후 실행한다. 틀린 경우에는 정답만 바꾸지 말고 어떤 연산에서 위치와 값을 혼동했는지 확인한다.

완성된 코드를 읽는 실습

```
letters = ["A", "B", "C", "D"]
letters.append("G")
letters.insert(1, "A-1")
letters.remove("C")
last = letters.pop()
print(last)
print(letters)
print(len(letters))
```

예상할 세 가지

last에 들어 있는 값, 남은 리스트, 원소의 개수를 각각 적는다.

Practice

해설: 추가·삭제가 끝난 뒤의 목록

실행한 작업	리스트의 상태
처음	['A', 'B', 'C', 'D']
append("G")	['A', 'B', 'C', 'D', 'G']
insert(1, "A-1")	['A', 'A-1', 'B', 'C', 'D', 'G']
remove("C")	['A', 'A-1', 'B', 'D', 'G']
pop()	['A', 'A-1', 'B', 'D']

반환값과 남은 자료를 나누어 읽는다

pop()이 돌려준 G가 last에 들어가고, 리스트에는 네 원소가 남는다. 따라서 마지막 세 출력은 G, ['A', 'A-1', 'B', 'D'], 4이다.

위치 삭제로 바꾸면 무엇이 달라지는가

remove("C") 대신 del letters[2]를 실행하면 그 시점의 인덱스 2에 있는 B가 삭제된다. 같은 숫자나 글자를 보더라도 어떤 방식으로 대상을 지정했는지 확인한다.

Practice

실습 2. 가격 목록을 수정하고 합계 구하기

주어진 목록을 순서대로 갱신한다

[85000, 140000, 200000]에서 첫 가격을 5000으로 바꾼다. 15000을 추가하고 10000과 20000을 각각 추가한다.

삭제 기준을 확인한다

마지막에는 가격이 140000인 원소를 제거한다. 인덱스 140000을 지우는 것이 아니라 그 값을 찾는 작업이다.

완료 후 세 결과를 확인한다

최종 가격 목록, 원소 개수, 합계를 각각 출력한다. 추가할 가격 두 개가 안쪽 리스트 하나로 들어가지 않도록 한다.

완성할 코드

```
prices = [85000, 140000, 200000]
prices[0] = _______
prices.append(_______)
prices.____([10000, 20000])
prices.remove(_______)
print(prices)
print(len(prices))
print(sum(prices))
```

실행 전에 확인할 내용

첫 원소 수정은 길이를 바꾸지 않는다. append, extend, remove를 실행할 때마다 길이는 얼마나 달라지는가?

Practice

해설 2. 목록·개수·합계를 서로 구분하기

In [3]:
```
prices = [85000, 140000, 200000]
prices[0] = 5000
prices.append(15000)
prices.extend([10000, 20000])
prices.remove(140000)
print(prices)
print(len(prices))
print(sum(prices))
```

```
[5000, 200000, 15000, 10000, 20000]
5
250000
```

최종 목록은 다섯 개의 숫자로 구성되며 합계는 250000이다. extend는 두 원소를 추가하고, remove는 가격이 140000인 원소 하나를 삭제한다.

Practice

실습 3. 원본을 유지하면서 상위 세 점수 구하기

정렬 기준과 원본 보존을 정한다

점수 [70, 95, 80, 95, 60]을 높은 순서로 읽으려 한다. 입력 순서의 scores도 이후에 다시 사용할 수 있어야 한다.

정렬 다음에 범위를 선택한다

먼저 내림차순으로 정렬한 새 리스트를 만들고, 그 결과의 앞 세 원소를 선택한다. 95가 두 번 있어도 서로 다른 두 관측값이다.

선택된 점수의 평균을 계산한다

전체 다섯 점수가 아니라 상위 세 점수의 합을 세 개수로 나눈다. 코드 실행 전에 top3와 평균을 예상한다.

완성할 코드

```
scores = [70, 95, 80, 95, 60]
ranked = sorted(scores, _____)
top3 = ranked[____]
print(scores)
print(ranked)
print(top3)
print(sum(top3) / len(top3))
```

실행 전에 확인할 내용

sorted의 reverse 인수와 [:3]을 이용한다. 같은 점수를 하나만 남기는 작업은 요구하지 않았다.

Practice

해설 3. 상위 점수의 선택과 원본 보존

In [7]:

```
scores = [70, 95, 80, 95, 60]
ranked = sorted(scores, reverse=True)
top3 = ranked[:3]
print(scores)
print(ranked)
print(top3)
print(sum(top3) / len(top3))
```

```
[70, 95, 80, 95, 60]
[95, 95, 80, 70, 60]
[95, 95, 80]
90.0
```

ranked는 [95, 95, 80, 70, 60], top3는 [95, 95, 80]이다. 평균은 90.0이고 scores는 처음 입력 순서를 유지한다.

Strings

문자열도 위치와 범위를 가진 시퀀스이다

문자열의 인덱싱도 0부터 시작한다

word = "abcdef"에서 word[0]은 a, word[5]는 f이다. 리스트와 같은 방식으로 음수 인덱스와 슬라이싱을 사용할 수 있다.

슬라이싱 결과는 문자열이다

"Python"[0:2]는 "Py", [2:5]는 "tho"이다. 리스트 슬라이싱이 리스트를 만드는 것과 달리 문자열을 선택한 결과는 문자열이다.

문자열의 원소는 직접 바꿀 수 없다

word[0] = "A"는 TypeError를 만든다. 문자열은 불변 자료형이므로 새로운 문자열을 만들고 그 결과를 이름에 다시 연결해야 한다.

문자열 접근과 슬라이싱

```
word = "abcdef"
print(word[0], word[5])
word = "Python"
print(word[0:2])
print(word[2:5])
print("J" + word[1:])
```

실행 결과

```
a f
Py
tho
Jython
```

연결과 수정의 차이

마지막 식은 새 문자열을 계산할 뿐 word에 다시 대입하지는 않는다.

Strings

그림으로 연결하기: 문자도 위치와 범위를 가진다

a	b	c	d	e	f
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1







같은 인덱싱 규칙을 다른 시퀀스에 적용한다

word = "abcdef"에서 0과 -6은 모두 a를 가리킨다. 문자열을 슬라이싱하면 부분 문자열이 만들어지며, 원본의 일부가 실제로 잘려 나가는 것은 아니다.

Strings

문자열 비교와 split()의 결과

비교는 처음으로 다른 문자에서 결정된다

"apple" < "orange"는 참이다. Python의 문자열 비교는 문자 코드 순서에 따른 사전식 비교이며, 모든 언어의 사전 정렬과 같다고 볼 수는 없다.

split()은 문자열을 여러 문자열로 나눈다

인수 없이 split()을 호출하면 공백류를 기준으로 단어를 나눈 리스트를 얻는다. 원래 문자열 s 자체를 리스트로 바꾸는 것은 아니다.

구두점까지 자동으로 없애지는 않는다

문장 끝의 today.에는 마침표가 포함된다. 단어 개수를 세거나 뒤에서 딕셔너리로 단어를 셀 때 어떤 문자열이 한 원소인지 먼저 확인한다.

비교 예

```
a = "apple"
b = "orange"
print(a < b)
```

출력: True

문자열을 단어 목록으로

```
s = ("Never put off till tomorrow "
     "what you can do today.")
words = s.split()
print(len(words))
print(words[0], words[-1])
```

출력: 10
Never today.

Strings

실행 화면: 단어 비교와 split() 결과

In [9]:

```
a, b = "apple", "orange"
print(a < b)
print("apple" == "apple")
print("Apple" == "apple")

s = "Never put off till tomorrow what you can do today."
words = s.split()
print(len(words), words[0], words[-1])
```

```
True
True
False
10 Never today.
```

문자열의 대소문자도 비교 대상이다. split()으로 얻은 마지막 단어 today.에는 마침표가 남아 있다. 원소 개수와 각 원소에 들어 있는 문자를 함께 본다.

Practice

실습 4. 문장을 단어 목록으로 바꾸고 일부 선택하기

문자열과 리스트를 구분한다

문장 "red blue red green"을 공백 기준으로 나누어 words에 저장한다. 결과는 문자열 하나가 아니라 문자열 네 개를 담은 리스트이다.

목록에서 필요한 부분을 읽는다

단어 개수, 처음 두 단어, 전체 역순 목록을 차례로 출력한다. red가 두 번 나오는 것을 오류나 중복 제거 대상으로 보지 않는다.

입력 문장의 변화 여부를 확인한다

마지막에 sentence를 출력한다. split과 슬라이싱은 원래 문자열을 수정하는 작업이 아니라 새 결과를 만드는 작업이다.

완성할 코드

```
sentence = "red blue red green"
words = sentence._____()
print(words)
print(len(words))
print(words[____])
print(words[____])
print(sentence)
```

실행 전에 확인할 내용

단어 순서를 거꾸로 하는 것과 문장의 모든 글자를 거꾸로 하는 것은 같은 결과일까?

Practice

해설 4. 단어 단위의 슬라이싱을 확인한다

In [8]:

```
sentence = "red blue red green"
words = sentence.split()
print(words)
print(len(words))
print(words[:2])
print(words[::-1])
print(sentence)
```

```
['red', 'blue', 'red', 'green']
4
['red', 'blue']
['green', 'red', 'blue', 'red']
red blue red green
```

words[:2]는 red와 blue를, words[::-1]은 green부터 시작하는 단어 목록을 만든다.
sentence[::-1]처럼 문자열 자체를 뒤집는 것과 다르다.

Nested Lists

2차원 리스트는 행 리스트들의 모음이다

바깥 리스트에서 먼저 행을 선택한다

s[1]은 두 번째 행인 [6, 7, 8, 9, 10]이다. s[1][2]는 그 행 안의 세 번째 원소이므로 8이 된다.

길이가 무엇을 세는지 구분한다

len(s)는 바깥 원소인 행의 개수 3이다. len(s[0])은 첫 번째 행의 원소 개수 5이다. 전체 숫자의 개수와 바로 같지는 않다.

두 번의 인덱싱도 같은 원리를 따른다

s[1][2] = 80은 두 번째 행의 세 번째 값을 바꾼다. 중첩 리스트는 행마다 길이가 다를 수도 있으므로 항상 직사각형이라고 가정하지 않는다.

3행 5열 예

```
s = [
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15]
]
print(s[1])
print(s[1][2])
print(len(s), len(s[0]))
```

실행 결과

```
[6, 7, 8, 9, 10]
8
3 5
```



Nested Lists

표의 행과 열을 두 번의 인덱싱으로 읽기

학생	국어	영어	수학	과학	사회
김철수	1	2	3	4	5
김영희	6	7	8	9	10
최자영	11	12	13	14	15

s[1][2]는 두 번째 학생의 세 번째 과목이다

먼저 바깥 리스트의 인덱스 1로 [6, 7, 8, 9, 10]을 고른다. 이어서 그 행의 인덱스 2를 읽으므로 결과는 8이다.

Nested Lists

행 선택과 원소 변경을 구분해서 보기

In [1]:
```
s = [[1, 2, 3, 4, 5],
     [6, 7, 8, 9, 10],
     [11, 12, 13, 14, 15]]
print(s[1])
print(s[1][2])
print(len(s), len(s[0]))
```

```
[6, 7, 8, 9, 10]
8
3 5
```

In [2]:
```
s[1][2] = 80
for row in s:
    print(row)
```

```
[1, 2, 3, 4, 5]
[6, 7, 80, 9, 10]
[11, 12, 13, 14, 15]
```

s[1]로 행을 읽은 뒤 [2]로 그 행의 원소를 선택한다. 마지막 셀의 80은 s[1][2]에 새 값을 대입한 결과이다. 반복문은 바깥 리스트의 행을 하나씩 출력한다.

Practice

실습 5. 각 행의 합과 한 과목의 평균 구하기

먼저 바깥 반복의 대상을 정한다

s의 원소는 숫자가 아니라 행 리스트이다. for row in s로 반복하면 첫 반복에서 [1, 2, 3, 4, 5]가 row에 들어온다.

한 열은 각 행에서 같은 위치를 고른다

세 번째 과목은 모든 행에서 인덱스 2를 읽어야 한다. 첫 학생의 세 번째 값과 두 번째 학생의 세 번째 값을 구분한다.

행 합계와 열 평균을 나누어 검증한다

행의 합을 세 줄로 출력하고 세 번째 과목의 값 세 개와 평균을 추가로 출력한다. 아래 수치는 구조를 설명하기 위한 예제이다.

완성할 코드

```
s = [[1, 2, 3, 4, 5],
     [6, 7, 8, 9, 10],
     [11, 12, 13, 14, 15]]
for row in s:
    print(____(row))
math_scores = [s[0][__],
               s[1][__], s[2][__]]
print(math_scores)
print(sum(math_scores) / len(math_scores))
```

실행 전에 확인할 내용

len(s)는 행 개수 3이다. 첫 행의 열 개수는 len(s[0])으로 확인한다.

Practice

해설 5. 행을 모으는 방향과 열을 모으는 방향

In [10]:
```
s = [[1, 2, 3, 4, 5],
     [6, 7, 8, 9, 10],
     [11, 12, 13, 14, 15]]
for row in s:
    print(sum(row))
math_scores = [s[0][2], s[1][2], s[2][2]]
print(math_scores)
print(sum(math_scores) / len(math_scores))
```

```
15
40
65
[3, 8, 13]
8.0
```

세 행의 합은 15, 40, 65이다. 세 번째 과목의 값은 [3, 8, 13]이고 평균은 8.0이다. 같은 표라도 행과 열 중 무엇을 묶는지에 따라 계산 대상이 달라진다.

Chapter Overview

리스트와 문자열을 설명할 때 확인할 세 가지

선택한 결과의 자료형

인덱스 하나는 원소를 읽고, 슬라이싱은 같은 종류의 시퀀스 일부를 만든다. 리스트 안에 문자열이 있을 때 두 결과의 모양을 구분한다.

원본의 변화와 반환값

append, insert, remove, reverse, sort는 원본을 바꾼다. pop은 제거한 값을 돌려주며 sorted는 정렬된 새 리스트를 만든다.

반복에서 읽는 하나의 단위

1차원 리스트의 반복은 원소 하나를 읽는다. 2차원 리스트에서는 행 리스트 하나를 읽는다. 다음 시간에는 위치 이외의 기준으로 자료를 다룬다.